

Buổi 14, bản Python

Nhập môn sân chơi lập trình thi đấu

Bản Python: kiến thức, mười bài tập và mười code mẫu

Ngày 17 tháng chín năm 2026

TS. Đỗ Phúc Hào

Phần	Nội dung
I	Kiến thức, viết riêng cho Python
II	Mười đề bài A tới J, giống hết bản C++
III	Gợi ý và lời giải chi tiết
IV	Mười code mẫu, lấy thẳng từ file đã được chấm

Bản Python, dùng chung đề bài và bộ test với bản C++

Đề bài trong tài liệu này giống hết bản C++, và mười bài được chấm trên đúng một bộ test. Phần kiến thức, lời giải và mã nguồn thì viết riêng cho Python, vì cạm bẫy là chuyện của ngôn ngữ chứ không phải của đề bài.

Phần I. Kiến thức

Buổi này không dạy thuật toán nào. Mười bài dưới đây gần như không cần ý tưởng, và mỗi bài chỉ nhằm đúng một lỗi cơ chế làm người mới mất điểm oan.

Nếu bạn đã đọc bản C++ thì ba mục lớn của nó đã biến mất ở đây: `trần số`, `getline` sau `>>`, và ép kiểu trước khi nhân. Python không có những thứ đó. Đổi lại nó có bốn thứ khác mà C++ không có, và ba trong bốn thứ ấy nguy hiểm hơn, vì chúng không hề báo lỗi, chúng chỉ lặng lẽ cho ra một đáp án khác.

1. Một bài thi trông như thế nào, và năm kết luận của máy chấm

Một bài gồm đề, giới hạn, dữ liệu vào, kết quả, và vài ví dụ. Bạn nộp một chương trình đọc từ đầu vào chuẩn và in ra đầu ra chuẩn. Máy chấm chạy nó trên hàng chục bộ test bạn không nhìn thấy, rồi trả về một trong năm kết luận:

Viết tắt	Nghĩa	Thường là do
AC	đúng hết	
WA	sai kết quả	thuật toán sai, hoặc đúng mà lệch một ở biên
TLE	quá thời gian	sai độ phức tạp, hoặc viết đúng mà viết chậm
RE	lỗi khi chạy	chỉ số ra ngoài, chia cho không, đệ quy quá sâu
MLE	quá bộ nhớ	dùng danh sách lớn hơn bạn tưởng

Dòng TLE là dòng bạn phải đọc kỹ hơn người học C++. Trong C++, TLE gần như luôn có nghĩa là thuật toán sai. Trong Python thì không: một thuật toán **đúng** vẫn có thể TLE chỉ vì cách viết, và bài H với bài J của buổi này tồn tại để bạn gặp chuyện đó một lần trong hoàn cảnh an toàn.

2. Khung chương trình chuẩn

```
import sys

def main():
    du_lieu = sys.stdin.buffer.read().split()
    # ...
    sys.stdout.write(ket_qua)

main()
```

Ba chi tiết trong khung này đều có lý do, và không cái nào là thói quen trang trí.

Vì sao học trong `main()` thay vì viết thẳng. Biến ở mức module là biến toàn cục, và CPython tra cứu biến toàn cục qua một từ điển còn biến địa phương qua một ô nhớ đánh số. Đo được: cùng một vòng lặp cộng dồn, đặt trong hàm nhanh hơn đặt ở mức module chừng một phần ba. Đây là dòng lệnh rẻ nhất trong cả buổi học.

Vì sao `sys.stdin.buffer.read()` thay vì `input()`. Đo trên chính file test lớn nhất của kho này, 8,2 MB: đọc cả luồng rồi `.split()` mất 0,183 giây, còn đọc từng dòng mất 0,364 giây. Gấp đôi. Với bài có giới hạn một giây thì riêng khoản này đã ăn một phần năm ngân sách mà chưa tính toán gì.

Vì sao `sys.stdout.write` thay vì `print` nhiều lần. `print` gọi một lần cho mỗi dòng. Gom kết quả vào danh sách rồi nối bằng `"\n".join(...)` và ghi một lần là cách duy nhất để in $2 \cdot 10^5$ dòng mà không mất nửa giây cho riêng việc in.

3. Ba tình huống đọc dữ liệu và ba cách đọc

Đề cho	Cách đọc	Ghi chú
các số, không quan tâm chúng nằm trên dòng nào	<code>sys.stdin.buffer.read().split()</code>	dùng được cho gần hết mọi bài
các dòng văn bản, dấu cách có ý nghĩa	<code>sys.stdin.buffer.read().split(b"\n")</code>	không dùng <code>.strip()</code> , nó ăn mất dấu cách
đọc tới hết dữ liệu, không biết trước bao nhiêu	vẫn là <code>read().split()</code>	độ dài danh sách chính là số phần tử

Tình huống thứ ba là bài C. Trong C++ nó là cái bẫy `while (!cin.eof())` đọc thừa một lần. Python có cái bẫy tương ứng nhưng khác hình: `input()` khi hết dữ liệu **ném EOFError** chứ không trả về chuỗi rỗng, nên vòng lặp phải bọc `try`. Đọc cả luồng một lần thì cả hai cái bẫy đều không có chỗ tồn tại.

4. Ba thứ bản C++ dạy ở mục này mà Python không có

Ghi lại cho rõ, vì nếu bạn học từ tài liệu C++ hoặc từ người dạy C++ thì bạn sẽ đi tìm những thứ này và không thấy.

Trần số. Số nguyên Python không có độ rộng cố định. $10^{**}9 * 10^{**}9$ cho ra đúng 10^{18} , và $2^{**}200$ cũng ra đúng. Toàn bộ mục 5 và mục 6 của bản C++, cộng cái bẫy của bài B, không còn đối tượng. Bài B ở đây đổi sang nhầm một lỗi khác: hai số nằm trên **một** dòng, nên `int(input())` sẽ ném `ValueError`.

getline sau >>. Không có hai kênh đọc chồng nhau nên không có chuyện dòng thừa. Bài D đổi sang nhầm chuyện khác: `readline()` **giữ lại** ký tự xuống dòng, nên `len()` lớn hơn độ dài thật đúng một, và bài này đếm ký tự.

Ép kiểu. Không có `(long long)a * b`. Nhưng có một chuyện ngược lại mà C++ không có, và nó ở mục kế tiếp.

5. / và //: một dấu gạch, hai phép toán khác hẳn nhau

Đây là thứ thay chỗ cho trần số trong lộ trình, và nó bẫy theo **hướng ngược lại** với C++.

```
7 / 2      # 3.5      chia thuc, LUON ra so thuc
7 // 2     # 3        chia lay phan nguyen
-7 // 2    # -4       lam tron XUONG, khong phai ve phia 0
```

Ba dòng trên là ba bắt ngờ riêng biệt.

Dòng đầu: trong C++, `7 / 2` với hai số nguyên cho ra 3. Người quen C++ viết `7 / n` rồi tưởng mình đang chia nguyên. Trong Python nó ra số thực, và nếu bạn đang cộng dồn tiền hoặc đếm thì kết quả sẽ mang theo sai số dấu chấm động.

Dòng hai: người quen Python viết `//` cho chắc, và ở bài E thì đó chính là lỗi, vì bài E cần phần thập phân.

Dòng ba là chỗ khó chịu nhất. `-7 // 2` cho `-4` chứ không phải `-3`, vì Python làm tròn xuống. C++ làm tròn về phía không. Cùng một biểu thức, hai ngôn ngữ, hai đáp án. Chuyện này xuất hiện lại ở mọi bài có toạ độ âm hoặc có chia đôi khoảng.

Quy tắc dùng được: **hỏi mình cần số nguyên hay số thực, rồi viết đúng dấu ấy, đừng viết theo thói quen.**

6. In số thực

`print(7 / 3)` cho ra `2.3333333333333335`. Đó không phải sáu chữ số của bất cứ thứ gì, và máy chấm sẽ trả về WA.

```
print("%.6f" % x)      # exactly six digits after the point
print(f"{x:.6f}")      # same thing, newer spelling
```

Và như bản C++ đã dạy: **chữ số có nghĩa khác chữ số thập phân**. `%.6g` cho sáu chữ số có nghĩa, `%.6f` cho sáu chữ số sau dấu chấm. Đề nói cái nào thì viết cái ấy.

7. Đọc giới hạn để chọn cách, kèm bảng tra đo trên Python

Bản C++ có một bảng tra nói khoảng 10^8 phép tính đơn giản mỗi giây. **Con số ấy sai hẳn một bậc nếu bạn viết Python**, và tin vào nó là cách chắc chắn nhất để chọn nhầm thuật toán.

Đo thật trên máy dựng kho này, Python 3.13:

Việc, lặp trong vòng for	Phép mỗi giây
<code>s += i</code> , thân đơn giản nhất có thể	35 triệu
thân có thêm một lệnh <code>if</code>	29 triệu
gọi một hàm mỗi vòng	22 triệu
đọc ghi phần tử list theo chỉ số	16 triệu
deque thêm vào rồi lấy ra	16 triệu
ghi vào dict	9,6 triệu
thân có nhân và chia dư, <code>s += i*i % p</code>	7,8 triệu
<code>sum(range(n))</code> , vòng lặp chạy trong mã C	88 triệu

Hai điều đáng rút ra, và điều thứ hai mới là điều lớn.

Thứ nhất, **thân vòng lặp quyết định nhiều hơn số vòng lặp**. Từ 35 triệu xuống 7,8 triệu là 4,5 lần, chỉ vì thêm một phép nhân và một phép chia dư. Nên đừng đếm số vòng, hãy đếm số phép tính thật.

Thứ hai, dòng cuối bảng nhanh hơn dòng đầu **11 lần** mà làm đúng một việc. Khác nhau ở chỗ vòng lặp của `sum` chạy bên trong mã C còn vòng lặp của bạn chạy trong trình thông dịch. Đây là ý trung tâm của việc viết Python thi đấu, và nó sẽ quay lại ở mọi buổi sau.

Bảng chọn cách, lấy ngân sách an toàn là 10^7 phép thông dịch cho một giây:

n	Độ phức tạp còn kịp
≤ 10	$O(n!)$
≤ 20	$O(2^n)$
≤ 300	$O(n^3)$
$\leq 3 \cdot 10^3$	$O(n^2)$
$\leq 3 \cdot 10^5$	$O(n \log n)$
$\leq 3 \cdot 10^6$	$O(n)$ viết thuần
$\leq 10^8$	$O(n)$ nhưng đẩy được vào <code>sum</code> , <code>map</code> , lát cắt

So với bảng C++ thì mỗi dòng tụt xuống chừng một bậc. Bài H của buổi này chính là chỗ bảng ấy phải được dùng: $n = q = 2 \cdot 10^5$, nhân vào là $4 \cdot 10^{10}$, và bảng nói ngay rằng cách hiển nhiên không thể kịp.

8. Danh sách, và cái bẫy bảng hai chiều

```
bang = [[0] * m] * n      # WRONG: n references to ONE row
bang = [[0] * m for _ in range(n)] # right: n independent rows
```

Dòng đầu tạo **một** hàng rồi lặp lại n lần cái tham chiếu tới nó. Gán `bang[0][0] = 5` thì cả n hàng cùng đổi. Nó in ra trông như một bảng, nó có đúng n hàng, `len()` đúng, và nó chỉ sai khi bạn ghi vào. Không có cảnh báo nào cả.

Đây là bẫy của bài G, và nó thuộc loại nguy hiểm nhất trong Python: **sai lặng lẽ, và trông đúng**.

9. Xâu, byte, và ký tự

Python không có kiểu ký tự riêng. Lặp trên một `str` cho ra các `str` dài một; lặp trên một `bytes` cho ra các **số nguyên**. Cả hai đều dùng được, nhưng đừng trộn.

```
for c in "abc":    print(c)    # 'a' 'b' 'c'
for c in b"abc":  print(c)    # 97 98 99
```

Các hàm `str.isupper()`, `str.islower()`, `str.isdigit()` hiểu cả Unicode, nên `'²'.isdigit()` là `True`. Nếu đề nói ASCII in được thì hãy so theo khoảng mã, đừng dựa vào chúng.

Bẫy của bài I là dòng văn bản **có dấu cách và có thể rỗng**. `input().split()` mất dấu cách, còn `input()` trên file không còn dòng nào sẽ ném `EofError`.

10. Chọn đúng cấu trúc, và bảng độ phức tạp phải thuộc

Việc	list	deque	dict / set
thêm vào cuối	$O(1)$	$O(1)$	
lấy ra ở cuối	$O(1)$	$O(1)$	
lấy ra ở đầu	$O(n)$	$O(1)$	
chèn vào đầu	$O(n)$	$O(1)$	
tìm một giá trị	$O(n)$	$O(n)$	$O(1)$
lấy theo chỉ số	$O(1)$	$O(n)$	

Dòng in đậm là bài J. `hang.pop(0)` trông y hệt `hang.pop()` nhưng nó dịch mọi phần tử còn lại xuống một ô. Đo được trên test lớn nhất của bài J: bản dùng `deque` mất 104 ms, bản dùng `list.pop(0)` mất 1112 ms, chậm hơn **10,7 lần**, mà hai bản khác nhau đúng một từ.

Dòng "tìm một giá trị" là bài H. `a.count(x)` và `x in a` đều là quét cả dãy.

11. Đọc đề: bốn chỗ luôn phải gạch chân

- Giới hạn.** Nhân chúng với nhau trước khi viết dòng lệnh đầu tiên.
- Biên.** "Kể cả hai đầu", "khác nhau đôi một", "có thể rỗng". Bài F và bài J sống bằng ba chữ.
- Định dạng kết quả.** Bao nhiêu chữ số, mỗi kết quả một dòng hay cùng một dòng.
- Trường hợp suy biến.** $n = 1$, dãy toàn âm, hàng đợi rỗng, dòng rỗng. Bốn thứ này có mặt trong bốn bài của buổi.

12. Khi chạy sai thì làm gì, bốn bước trước khi hỏi người khác

- Chạy lại ví dụ trong đề bằng tay, trên giấy, không chạy máy.
- Thử ca nhỏ nhất mà đề cho phép, rồi ca suy biến ở mục 11.
- In ra giữa chừng bằng `print(..., file=sys.stderr)`, vì `stderr` không bị máy chấm so.
- Đọc lại đúng bốn chỗ ở mục 11.

Với TLE thì thêm một bước riêng của Python, và nó đứng **trước** cả bốn bước trên: hỏi xem vòng lặp trong cùng của bạn có chạy trong trình thông dịch không, và có đẩy được vào một hàm dựng sẵn không.

13. Cạm bẫy và bộ khung tự hỏi

Cạm bẫy	Dấu hiệu
<code>mx = 0</code> cho dãy có thể toàn âm	đáp án ra 0 ở test có số âm
<code>//</code> khi đề cần phần thập phân	đáp án luôn tròn
<code>/</code> khi đề cần số nguyên	đáp án có <code>.0</code> ở cuối
<code>[[0]*m]*n</code>	mọi hàng giống hệt nhau
<code>list.pop(0)</code>	đúng nhưng TLE ở test lớn
<code>a.count(x)</code> trong vòng truy vấn	đúng nhưng TLE ở test lớn
<code>input()</code> trong vòng lặp lớn	chậm gấp đôi mà không rõ vì sao
<code>.strip()</code> khi dấu cách có nghĩa	lệch ở test có dấu cách đầu cuối
<code>str.isdigit()</code> với dữ liệu không thuần ASCII	đếm dư

Bộ khung tự hỏi, dùng cho mọi bài từ buổi này trở đi:

1. Giới hạn nhân vào là bao nhiêu? Đối chiếu bảng ở mục 7.
2. Cần số nguyên hay số thực? Viết `//` hay `/`?
3. Có chỗ nào lấy ra ở đầu danh sách không? Có thì dùng deque.
4. Có chỗ nào tìm kiếm trong vòng lặp không? Có thì dùng dict hay set.
5. Vòng lặp trong cùng có đẩy được vào sum, map, join, hay lát cắt không?

Phần II. Mười bài tập

Giới hạn thời gian ghi trong `py/gioi-han.json` cho từng bài. Tự chấm từ thư mục gốc của kho:

```
python tools/cham-py.py topics/14-nhap-mon/py/solutions/ref/bai01.py
python tools/cham-py.py topics/14-nhap-mon --all
```

Bài	Tên	Học điều gì	Cạm bẫy nhắm tới	File nộp
A	Tổng và giá trị lớn nhất	kiểu số, khởi tạo	max khởi tạo bằng 0	bai01.py
B	Tích hai số	thứ tự nhân và gán	ép kiểu sau khi nhân	bai02.py
C	Đọc tới hết dữ liệu	vòng lặp đọc	dùng <code>eof()</code>	bai03.py
D	Dòng dài nhất	<code>getline</code> sau <code>>></code>	quên <code>cin.ignore()</code>	bai04.py
E	Trung bình cộng	định dạng số thực	quên <code>fixed</code>	bai05.py
F	Đếm trong đoạn	từ chỉ biên trong đề	dùng <code><</code> thay <code><=</code>	bai06.py
G	Tổng từng hàng	mảng hai chiều	nhầm hàng với cột	bai07.py
H	Đếm số lần xuất hiện	đọc giới hạn chọn cách	hai vòng lồng nhau, TLE	bai08.py
I	Phân loại ký tự	xâu và ký tự	dùng <code>cin >> s</code>	bai09.py
J	Mô phỏng hàng đợi	đọc đề tới hết	bỏ sót ca rỗng	bai10.py

Cạm bẫy của **bản Python**, khác hẳn bảng của bản C++:

Bài	Tên	Học điều gì	Cạm bẫy nhắm tới	File nộp
A	Tổng và giá trị lớn nhất	khởi tạo giá trị lớn nhất	<code>mx = 0</code> với dãy toàn số âm	bai01.py
B	Tích hai số	đọc hai số trên một dòng	<code>int(input())</code> trên dòng có hai số	bai02.py
C	Đọc tới hết dữ liệu	dừng đúng lúc hết dữ liệu	<code>input()</code> ném <code>EOFError</code> chứ không trả về rỗng	bai03.py
D	Dòng dài nhất	đọc nguyên dòng, giữ dấu cách	<code>getline()</code> giữ <code>\n</code> nên <code>len</code> lệch một	bai04.py
E	Trung bình cộng	<code>/</code> và <code>//</code> khác nhau	dùng <code>//</code> nên mất phần thập phân	bai05.py
F	Đếm trong đoạn	từ chỉ biên trong đề	dùng <code><</code> thay <code><=</code>	bai06.py

G	Tổng từng hàng	dựng bảng hai chiều	<code>[[0]*m]*n</code> cho n tham chiếu tới một hàng	bai07.py
H	Đếm số lần xuất hiện	nhân hai giới hạn trước khi viết	<code>a.count(x)</code> mỗi truy vấn là quét cả dãy	bai08.py
I	Phân loại ký tự	dòng có dấu cách, và có thể rỗng	<code>input().split()</code> mất dấu cách	bai09.py
J	Mô phỏng hàng đợi	chọn đúng cấu trúc	<code>list.pop(0)</code> là $O(n)$, phải dùng deque	bai10.py

Bài A. Tổng và giá trị lớn nhất

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên. Hãy in tổng của cả dãy và giá trị lớn nhất trong dãy.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)

Kết quả Một dòng gồm hai số cách nhau bởi dấu cách: tổng và giá trị lớn nhất.

Ví dụ

Input	Output
5	1 4
3 -1 -7 2 4	

Giải thích. Tổng bằng $3 - 1 - 7 + 2 + 4 = 1$. Giá trị lớn nhất là 4.

Lưu ý. Hãy tự thử dãy $-5 -2 -9$. Nếu chương trình của bạn in ra 0 thì bạn đang khởi tạo giá trị lớn nhất bằng 0.

Bài B. Tích hai số

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho hai số nguyên a và b . Hãy in tích của chúng.

Dữ liệu vào

- Dòng 1: hai số nguyên a và b ($|a|, |b| \leq 10^9$)

Kết quả một số nguyên.

Ví dụ

Input	Output
1000000000 1000000000	1000000000000000000

Giải thích. Cả hai thừa số lên tới 10^9 và tích là 10^{18} . Trong Python điều đó hoàn toàn không sao: số nguyên không có độ rộng cố định nên phép nhân luôn cho kết quả đúng, kể cả với số lớn hơn nữa.

Lưu ý. Bẫy của bài này không nằm ở phép nhân mà ở chỗ đọc. Hai số nằm trên **một** dòng, nên `int(input())` sẽ ném `ValueError`. Phải tách rồi mới đổi kiểu.

Bài C. Đọc tới hết dữ liệu

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Dữ liệu gồm một số nguyên chưa biết trước số lượng, ngăn cách nhau bởi dấu cách hoặc dấu xuống dòng. Hãy in ra số lượng và tổng của chúng.

Dữ liệu vào

- Các số nguyên cho tới hết dữ liệu (số lượng từ 1 tới $2 \cdot 10^5$, mỗi số $|x| \leq 10^9$)

Kết quả Một dòng gồm hai số cách nhau bởi dấu cách: số lượng và tổng.

Ví dụ

Input	Output
1	3 6
2	
3	

Giải thích. Có ba số, tổng bằng 6. Các số có thể nằm trên một dòng hay nhiều dòng tùy ý và điều đó không được làm đổi kết quả.

Lưu ý. `input()` khi hết dữ liệu ném **`EOFError`** chứ không trả về chuỗi rỗng, nên vòng lặp đọc từng dòng phải bọc try. Đọc cả luồng một lần thì cái bấy ấy không có chỗ tồn tại. Test 08 đặt cả $2 \cdot 10^5$ số trên đúng một dòng.

Bài D. Dòng dài nhất

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho n dòng văn bản. Hãy in ra dòng **dài nhất**, tính theo số ký tự và có tính cả dấu cách. Nếu có nhiều dòng cùng dài nhất thì in dòng **xuất hiện trước**.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 1000$)
- n dòng tiếp theo, mỗi dòng gồm 1 tới 100 ký tự là chữ cái Latin, chữ số hoặc dấu cách, không có dấu cách ở đầu và cuối dòng

Kết quả một dòng.

Ví dụ

Input	Output
3	mot hai ba bon
mot hai	
mot hai ba bon	
xin chao	

Giải thích. Ba dòng dài lần lượt 7, 14 và 8 ký tự.

Lưu ý. Đề đếm cả dấu cách, nên `.strip()` là sai: nó ăn mất chính thứ đang được đếm. Và `readline()` giữ lại ký tự xuống dòng, nên len lớn hơn độ dài thật đúng một. Hãy cắt đúng ký tự xuống dòng, không hơn.

Bài E. Trung bình cộng

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên. Hãy in trung bình cộng của chúng với **đúng sáu chữ số sau dấu chấm**.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^6$)

Kết quả một số thực in với đúng sáu chữ số sau dấu chấm.

Ví dụ

Input	Output
3	2.333333
1 2 4	

Giải thích. Tổng bằng 7, chia cho 3 được 2,333... và làm tròn tới sáu chữ số thập phân.

Lưu ý. Cộng dồn bằng số nguyên rồi mới chia ở bước cuối, và chia bằng `/` chứ không phải `//`. Người quen C++ và người quen Python mắc lỗi ngược nhau ở đây. In thì phải `("%.6f": print(tong / n)` cho 2.3333333333333335, không phải sáu chữ số của bất cứ thứ gì.

Bài F. Đếm trong đoạn

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và q truy vấn. Mỗi truy vấn cho hai số l và r ; hãy đếm xem trong dãy có bao nhiêu giá trị nằm trong đoạn từ l tới r , **kể cả hai đầu**.

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 1000$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)
- q dòng tiếp theo, mỗi dòng hai số l và r ($1 \leq r, |l|, |r| \leq 10^9$)

Kết quả Mỗi truy vấn in một dòng.

Ví dụ

Input	Output
5 3	3
1 5 2 4 3	1
1 3	0
5 5	
6 10	

Giải thích. Trong đoạn từ 1 tới 3 có ba giá trị là 1, 2, 3. Đoạn từ 5 tới 5 chỉ chứa giá trị 5. Không giá trị nào nằm từ 6 tới 10.

Lưu ý. Ba chữ "kể cả hai đầu" là toàn bộ bài này. Test 08 cho **mọi** đầu mút truy vấn đều là một giá trị có thật trong dãy.

Bài G. Tổng từng hàng

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho bảng n hàng và m cột gồm các số nguyên. Hãy in tổng của từng hàng, mỗi tổng trên một dòng, theo thứ tự từ hàng đầu tới hàng cuối.

Dữ liệu vào

- Dòng 1: n và m ($1 \leq n, m \leq 500$)
- n dòng tiếp theo, mỗi dòng m số nguyên ($|\text{giá trị}| \leq 10^9$)

Kết quả n dòng.

Ví dụ

Input	Output
2 3	6
1 2 3	15
4 5 6	

Giải thích. Hàng thứ nhất có tổng $1 + 2 + 3 = 6$, hàng thứ hai có tổng $4 + 5 + 6 = 15$.

Lưu ý. Nếu bạn dựng bảng bằng `[[0] * m] * n` thì cả n hàng dùng chung **một** hàng duy nhất, và nó chỉ sai khi bạn ghi vào. Test 08 là bảng chỉ có một hàng và 500 cột.

Bài H. Đếm số lần xuất hiện

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và q truy vấn. Mỗi truy vấn cho một giá trị x ; hãy đếm xem x xuất hiện bao nhiêu lần trong dãy.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)
- Dòng 3: q ($1 \leq q \leq 2 \cdot 10^5$)
- Dòng 4: q số nguyên x ($|x| \leq 10^9$)

Kết quả Mỗi truy vấn in một dòng.

Ví dụ

Input	Output
5	2
1 2 2 3 3	2
3	0
2 3 9	

Giải thích. Giá trị 2 xuất hiện hai lần, giá trị 3 xuất hiện hai lần, giá trị 9 không xuất hiện.

Lưu ý. Hãy nhân hai giới hạn với nhau **trước khi** viết vòng lặp. Nếu con số bạn nhận được lớn hơn 10^8 thì cách làm hiển nhiên sẽ nhận TLE chứ không phải WA.

Bài I. Phân loại ký tự

Giới hạn thời gian: 1 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho một dòng văn bản. Hãy đếm số chữ cái in hoa, số chữ cái in thường, số chữ số, và số ký tự thuộc loại khác.

Dữ liệu vào

- Dòng 1: một dòng văn bản dài từ 0 tới 10^5 ký tự, gồm các ký tự ASCII in được

Kết quả Một dòng gồm bốn số cách nhau bởi dấu cách: số chữ hoa, số chữ thường, số chữ số, số ký tự khác.

Ví dụ

Input	Output
Hello World 123!	2 8 3 3

Giải thích. Hai chữ hoa là H và W. Tám chữ thường là ello và orld. Ba chữ số là 123. Ba ký tự khác là hai dấu cách và một dấu chấm than.

Lưu ý. Dòng văn bản **có dấu cách**, nên `input().split()` sẽ mất chúng. Và đề cho phép dòng dài **0** ký tự, mà `input()` trên file không còn dòng nào thì ném `EOFError`.

Bài J. Mô phỏng hàng đợi

Giới hạn thời gian: 0,5 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Có một hàng người chờ, ban đầu rỗng. Hãy xử lý n thao tác, mỗi thao tác là một trong ba loại:

- IN <ten>: người tên <ten> xếp vào **cuối** hàng.
- OUT: người **đầu** hàng rời đi; in ra tên người đó. Nếu hàng đang **rỗng** thì in EMPTY.
- SIZE: in ra số người đang xếp hàng.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- n dòng tiếp theo, mỗi dòng một thao tác. Tên gồm 1 tới 20 chữ cái thường

Kết quả Mỗi thao tác OUT và SIZE in một dòng.

Ví dụ

Input	Output
6	an
IN an	1
IN binh	binh
OUT	EMPTY
SIZE	
OUT	
OUT	

Giải thích. Sáu thao tác lần lượt cho an, 1, binh, rồi EMPTY vì hàng đã rỗng.

Lưu ý. Đừng bỏ sót ca hàng rỗng. Và đừng lấy người đầu hàng ra bằng `list.pop(0)`: nó dịch mọi phần tử còn lại xuống một ô nên là $O(n)$. Dùng `collections.deque`.

Phần III. Gợi ý và lời giải

Mỗi bài có gợi ý nhỏ trước, lời giải đầy đủ sau. Hãy đọc gợi ý rồi quay lại bàn phím, đừng đọc thẳng xuống. Mã đầy đủ nằm ở phần IV.

Bài A. Tổng và giá trị lớn nhất

Gợi ý 1. Bạn không cần vòng lặp nào cả.

Gợi ý 2. Nếu bạn vẫn muốn viết vòng lặp, hãy hỏi: giá trị khởi tạo của biến giữ số lớn nhất là gì, và nó đúng với dãy -5 -2 -9 không?

Lời giải. `sum(a)` và `max(a)` làm cả hai việc, và cả hai đều chạy trong mã C nên nhanh hơn vòng lặp tự viết chừng vài lần. Quan trọng hơn: `max` trên danh sách **không cần giá trị khởi tạo**, nên cái bẫy của bài không có chỗ để tồn tại. Đây là lần đầu trong kho bạn gặp một quy tắc sẽ lặp lại rất nhiều: *cách viết làm cho lỗi không thể xảy ra tốt hơn cách viết rồi nhớ tránh lỗi*.

Nếu bạn buộc phải tự lập thì khởi tạo bằng `a[0]`, đừng bao giờ bằng 0.

Bài B. Tích hai số

Gợi ý 1. Đọc lại dòng dữ liệu vào: hai số nằm ở đâu?

Gợi ý 2. `int(input())` làm gì khi dòng đó là 1000000000 1000000000?

Lời giải. Nó ném `ValueError`, vì `int` không nhận một chuỗi có dấu cách. Phải tách rồi mới đổi: `a, b = map(int, input().split())`.

Còn phép nhân thì không có gì phải lo. $10^9 * 10^9 = 10^{18}$ và Python cho đúng kết quả, vì số nguyên của nó không có độ rộng cố định. Bản C++ của bài này tồn tại **chỉ** để dạy tràn số, và trong Python nó không còn dạy gì về số học nữa, nó dạy về đọc dữ liệu.

Đây là một bài đáng nhớ vì lý do khác: nó cho thấy dịch một khoá học sang ngôn ngữ khác không phải là dịch chữ. Cái bẫy đi mất, và phải thay bằng cái bẫy có thật.

Bài C. Đọc tới hết dữ liệu

Gợi ý 1. Đề nói các số có thể nằm trên một dòng hay nhiều dòng tùy ý. Cách đọc nào không quan tâm chuyện đó?

Gợi ý 2. Nếu bạn định viết `while True: x = int(input())` thì hãy chạy thử nó cho tới hết dữ liệu và xem nó dừng thế nào.

Lời giải. Nó không dừng, nó ném `E0FError`. Đó là hình dạng Python của cái bẫy `while (!cin.eof())` trong C++, và nó dễ mắc hơn bản C++ vì chương trình sập chứ không cho ra đáp án sai, tức bạn nhận RE chứ không phải WA và sẽ mất thời gian tìm nhầm chỗ.

Đọc cả luồng một lần thì không cần biết dữ liệu chia dòng ra sao, và số phần tử chính là độ dài danh sách:

```
du_lieu = sys.stdin.buffer.read().split()
print(len(du_lieu), sum(map(int, du_lieu)))
```

Đo thật: bản đọc từng dòng có bọc try chỉ chậm hơn 1,2 lần trên test lớn nhất của bài này, tức ở đây nó **không** phải chuyện tốc độ. Nó là chuyện đúng sai và chuyện gọn.

Bài D. Dòng dài nhất

Gợi ý 1. Đề nói tính cả dấu cách. Vậy `.strip()` có dùng được không?

Gợi ý 2. `sys.stdin.readline()` trả về cái gì ở cuối mỗi dòng, và nó ảnh hưởng `len()` thế nào?

Lời giải. Hai bấy nằm cạnh nhau và chúng kéo về hai hướng ngược nhau.

`readline()` giữ lại `"\n"`, nên `len` lớn hơn độ dài thật đúng một ký tự. Với mọi dòng thì lệch đều nhau nên so sánh vẫn ra đúng, **trừ dòng cuối cùng** nếu file không kết thúc bằng ký tự xuống dòng. Lúc ấy dòng cuối bị tính thiếu một so với các dòng khác, và nếu nó là dòng dài nhất thì bạn trượt.

Còn `.strip()` thì xóa cả dấu cách ở hai đầu, mà đề đang đếm dấu cách. Cách đúng là cắt đúng ký tự xuống dòng, không hơn: `.rstrip(b"\r\n")` hoặc tách sẵn bằng `.split(b"\n")`.

Điều kiện "nhiều dòng cùng dài nhất thì lấy dòng trước" được thỏa miễn là bạn dùng `>` chứ không phải `>=` khi cập nhật.

Bài E. Trung bình cộng

Gợi ý 1. Cộng bằng số nguyên hay số thực?

Gợi ý 2. Bạn định viết `tong // n` hay `tong / n`?

Lời giải. Cộng bằng số nguyên, chia ở bước cuối, và chia bằng `.`

Cộng bằng số thực thì sai số dấu chấm động tích lại qua 10^5 phép cộng. Chia bằng `//` thì mất sạch phần thập phân và bài hỏng hoàn toàn. Đây là chỗ người quen C++ và người quen Python mắc lỗi **ngược nhau**: người quen C++ viết `/` rồi tưởng mình đang chia nguyên, người quen Python viết `//` cho chắc rồi mất phần lẻ.

In thì phải `"%.6f". print(tong / n)` cho `2.3333333333333335`, không phải sáu chữ số của bất cứ thứ gì.

Bài F. Đếm trong đoạn

Gợi ý 1. Ba chữ "kể cả hai đầu" nằm ở đâu trong điều kiện của bạn?

Gợi ý 2. Python cho viết `l <= x <= r` thành một biểu thức. Hãy dùng nó.

Lời giải. n và q đều tối đa 1000 nên tích chỉ 10^6 , và bảng ở mục 7 nói cách hiển nhiên thừa sức kịp. Bài này hoàn toàn là chuyện đọc đề.

`sum(1 for x in a if l <= x <= r)` viết đúng như câu tiếng Việt. Đây là một trong vài chỗ hiểm hoi mà Python làm cho bản đúng **ngắn hơn** bản sai, nên hãy tận dụng: mỗi lần cách viết tự nhiên trùng với cách viết đúng thì bạn bớt được một chỗ để sai.

Test 08 cho mọi đầu mút truy vấn đều là một giá trị có thật trong dãy, tức nó nhắm thẳng vào ai viết `<` thay `<=`.

Bài G. Tổng từng hàng

Gợi ý 1. Bạn có thật sự cần dựng bảng hai chiều không?

Gợi ý 2. Nếu có, hãy thử `bang = [[0] * m] * n` rồi gán `bang[0][0] = 5` và in cả bảng ra xem.

Lời giải. Cả n hàng cùng đối, vì `[[0] * m] * n` lặp lại cái **tham chiếu** tới một hàng duy nhất chứ không tạo n hàng. Cách đúng là `[[0] * m for _ in range(n)]`.

Nhưng ở bài này thì cách tốt nhất là không dựng bảng: dữ liệu vào đã là một dãy phẳng, và hàng thứ i nằm ở lát cắt `du_lieu[2 + i*m : 2 + (i+1)*m]`. Không dựng bảng thì không có chỗ cho bẫy, đúng theo quy tắc đã nói ở bài A.

Bẫy nhầm hàng với cột thì Python và C++ giống nhau, và nó dễ thấy vì số dòng kết quả cũng sai theo.

Bài H. Đếm số lần xuất hiện

Gợi ý 1. Nhân n với q trước khi viết vòng lặp.

Gợi ý 2. `a.count(x)` là một lời gọi hàm, nhưng nó tốn bao nhiêu?

Lời giải. $2 \cdot 10^5 \times 2 \cdot 10^5 = 4 \cdot 10^{10}$. Bảng ở mục 7 nói ngân sách an toàn là 10^7 , tức cách hiển nhiên chậm hơn bốn nghìn lần.

`a.count(x)` quét cả dãy mỗi lần gọi. Nó **trông** như một phép tính vì nó là một lời gọi hàm dựng sẵn, và đó chính là chỗ nó nguy hiểm hơn hai vòng lồng nhau trong C++: bản C++ sai thì bạn nhìn thấy hai dấu ngoặc nhọn lồng nhau, còn bản Python sai thì ngắn hơn bản đúng.

Đếm một lần bằng Counter rồi tra:

```
dem = Counter(a)
... dem.get(x, 0) ...
```

Đo được: bản mẫu 110 ms, bản dùng count vượt 30 giây vẫn chưa xong, tức chậm hơn hơn 250 lần. Đây là một trong hai bài của buổi có bẫy tốc độ thật, và cổng 13 của kho dùng chính nó làm bằng chứng rằng giới hạn thời gian siết được một thứ có thật.

Một mẹo nhỏ có thật trong bản mẫu: giữ khoá ở dạng byte thô thay vì đổi sang số nguyên. Đề bảo đảm không có số nào viết kiểu 007, nên hai token bằng nhau khi và chỉ khi hai số bằng nhau, và ta tiết kiệm được $4 \cdot 10^5$ lần gọi int.

Bài I. Phân loại ký tự

Gợi ý 1. Dòng văn bản có dấu cách. `input().split()` giữ được chúng không?

Gợi ý 2. Đề nói dòng dài từ 0 ký tự. Chương trình của bạn làm gì với dòng rỗng?

Lời giải. Hai bẫy. `split()` mất dấu cách, và dòng rỗng làm `input()` ném EOFError nếu file không còn dòng nào. Đọc bằng `readline()` rồi cắt đúng ký tự xuống dòng thì cả hai đều không xảy ra: `readline()` trên hết dữ liệu trả về chuỗi rỗng chứ không ném.

Về việc phân loại: `str.isupper()` và họ hàng hiểu Unicode, nên chúng trả True cho những ký tự mà đề không coi là chữ. Đề nói ASCII in được, nên so theo khoảng mã là cách nói đúng điều đề nói.

Một điều thành thật. Khi đo lại để đặt giới hạn thời gian, bản viết bằng `str.isupper()` hoá ra **nhạy hơn** bản duyệt byte của tôi, 47 ms so với 73 ms. Bản mẫu vẫn giữ cách duyệt byte vì nó đúng nghĩa hơn với đề, nhưng đừng để ai nói với bạn rằng duyệt byte thì luôn nhanh hơn. Ở đây nó không.

Bài J. Mô phỏng hàng đợi

Gợi ý 1. Ba loại thao tác, và một trong ba loại có ca suy biến. Ca nào?

Gợi ý 2. Bạn định lấy người đầu hàng ra bằng lệnh gì, và lệnh ấy tốn bao nhiêu?

Lời giải. Ca suy biến là OUT khi hàng rỗng, phải in EMPTY. Bỏ sót nó thì chương trình sập ở test đầu tiên có ca ấy.

Còn chuyện tốc độ mới là bài học chính. `hang.pop(0)` trông y hệt `hang.pop()` nhưng nó dịch mọi phần tử còn lại xuống một ô, tức $O(n)$. Trên test lớn nhất của bài, hàng đợi dài nhất 9949 người và tổng số lượt dịch là 398.800.710.

Đo được: deque 104 ms, `list.pop(0)` 1112 ms. Chậm hơn **10,7 lần** vì đúng một từ.

Đây là bài đáng nhớ nhất buổi, vì trong C++ nó **không có bẫy nào cả**: `std::queue` là công cụ hiển nhiên và nó đã là $O(1)$. Cái bẫy này sinh ra từ chính Python, từ chỗ `list` làm được mọi việc nên người ta dùng nó cho mọi việc.

Và nó còn dạy một điều về giới hạn thời gian. 1112 ms vẫn lọt giới hạn 3000 ms mà tôi định đặt lúc đầu. Nếu để vậy thì bản sai vẫn đậu, và tài liệu sẽ dạy một cái bẫy mà không có cách nào chứng minh là nó bẫy được ai. Giới hạn của bài này vì thế đặt 500 ms, và con số ấy không phải chọn cho đẹp: nó nằm **giữa** 104 và 1112.

Phần IV. Mười code mẫu đầy đủ

Các khối dưới đây **sinh ra từ chính file đã được chấm**, không chép tay. Sửa `py/solutions/ref/baiNN.py` rồi chạy lại `tools/dung-note-py.py` thì phần này tự cập nhật, nên nó không thể lệch với thứ máy đã chạy.

Code bài A

```
# Bai A. Sum and maximum of a sequence.
#
# The trap is the same one C++ has: seeding the maximum with 0 makes every all-negative
# case wrong. Python offers a way out that C++ does not, and it is the right one here:
# max() over the list needs no seed at all, so the bug has nowhere to live.
import sys

def main():
    du_lieu = sys.stdin.buffer.read().split()
    n = int(du_lieu[0])
    a = list(map(int, du_lieu[1:1 + n]))
    print(sum(a), max(a))

main()
```

Code bài B

```
# Bai B. Product of two integers.
#
# In C++ this problem exists to teach integer overflow: int * int overflows before the
# result is ever assigned to a long long. Python has no fixed-width integers, so that
# lesson has no subject here and the arithmetic is simply correct.
#
# What is left is the reading itself: both numbers sit on ONE line, so int(input()) would
# raise ValueError. That is the Python-shaped mistake this problem now catches.
import sys

def main():
    a, b = map(int, sys.stdin.buffer.read().split()[:2])
    print(a * b)

main()
```

Code bài C

```
# Bai C. Read until end of input.
#
# The C++ version of this trap is while (!cin.eof()), which reads one item too many.
# Python's shape of the same mistake is `while True: x = int(input())`, because input()
# raises EOFError instead of returning a falsy value, so the loop needs a try block that
# people forget.
#
# Reading the whole stream at once sidesteps both, and it also answers the note in the
# statement: the numbers may be spread over any number of lines, and .split() does not
# care which.
import sys
```

```
def main():
    du_lieu = sys.stdin.buffer.read().split()
    print(len(du_lieu), sum(map(int, du_lieu)))

main()
```

Code bài D

```
# Bai D. Longest line.
#
# In C++ this problem exists for exactly one line of code: cin.ignore() after cin >> n,
# because >> leaves the newline in the buffer and the first getline returns empty.
#
# Python has no such coupling, but it has its own version of the same wound. Reading with
# sys.stdin.readline() keeps the trailing "\n", so len() is one too big on every line but
# possibly the last, and the comparison silently favours whichever line the file happened
# to end on. Strip exactly the newline and nothing else: the statement counts spaces, so
# .strip() would be wrong too, it would eat them.
import sys

def main():
    dong = sys.stdin.buffer.read().split(b"\n")
    n = int(dong[0])
    tot_nhat = b""
    for i in range(1, n + 1):
        d = dong[i].rstrip(b"\r") if i < len(dong) else b""
        if len(d) > len(tot_nhat):
            tot_nhat = d
    sys.stdout.buffer.write(tot_nhat + b"\n")

main()
```

Code bài E

```
# Bai E. Arithmetic mean, six digits after the point.
#
# Two separate traps, and Python only inherits one of them.
#
# Inherited: printing the raw float. print(7 / 3) gives 2.3333333333333335, which is not
# six digits of anything. Formatting is not optional.
#
# New, and this one is pure Python: `/` is float division and `//` is floor division. Use
# // here and 7 // 3 is 2, so the answer is 2.000000 and every test with a non-exact mean
# fails. C++ has the mirror image of this bug, where / on two ints truncates, so the two
# languages trap you in opposite directions with the same symbol.
import sys

def main():
    du_lieu = sys.stdin.buffer.read().split()
    n = int(du_lieu[0])
    tong = sum(map(int, du_lieu[1:1 + n]))
    print("%.6f" % (tong / n))

main()
```

Code bài F

```
# Bai F. Count values inside a closed range.
#
# "Ke ca hai dau" is the whole problem. Python spells the closed test the way the sentence
# reads, l <= x <= r, which is one of the few places the language makes the correct version
# shorter than the wrong one.
import sys

def main():
    du_lieu = sys.stdin.buffer.read().split()
    n = int(du_lieu[0])
    q = int(du_lieu[1])
    a = list(map(int, du_lieu[2:2 + n]))
    p = 2 + n
    ra = []
    for _ in range(q):
        l = int(du_lieu[p])
        r = int(du_lieu[p + 1])
        p += 2
        ra.append(sum(1 for x in a if l <= x <= r))
    sys.stdout.write("\n".join(map(str, ra)) + "\n")

main()
```

Code bài G

```
# Bai G. Row sums of a table.
#
# The C++ trap is swapping rows and columns. Python keeps that one and adds a worse one of
# its own: building the table with [[0] * m] * n gives n references to ONE row, so writing
# to bang[0][0] writes to every row at once. It looks like a table, it prints like a table
# until you modify it, and nothing warns you.
#
# Here the table is built by slicing the flat token list, so no aliasing is possible. The
# note is in the lesson, not only in this comment: see section 9 of the handout.
import sys

def main():
    du_lieu = sys.stdin.buffer.read().split()
    n = int(du_lieu[0])
    m = int(du_lieu[1])
    ra = []
    p = 2
    for _ in range(n):
        ra.append(sum(map(int, du_lieu[p:p + m])))
        p += m
    sys.stdout.write("\n".join(map(str, ra)) + "\n")

main()
```

Code bài H

```
# Bai H. Count occurrences, answering q queries.
#
# n and q are both 2*10^5, so the obvious nested loop is 4*10^10 operations. The statement
# tells you to multiply the two limits BEFORE writing the loop, and this is the problem
# where that habit pays.
#
# Python dresses the slow version up nicely: a.count(x) inside the query loop reads like
# one operation and is in fact a full scan. It is the same asymptotic mistake as the C++
```

```
# nested loop, but it is far easier to write by accident because it is shorter than the
# correct version, not longer.
import sys
from collections import Counter

def main():
    du_lieu = sys.stdin.buffer.read().split()
    n = int(du_lieu[0])
    dem = Counter(du_lieu[1:1 + n])
    q = int(du_lieu[1 + n])
    truy_van = du_lieu[2 + n:2 + n + q]
    # The keys stay as raw bytes: the tokens are compared, never arithmetic, so parsing
    # 4*10^5 integers would be work done for nothing. b"007" never appears in the input.
    sys.stdout.write("\n".join(str(dem.get(x, 0)) for x in truy_van) + "\n")

main()
```

Code bài I

```
# Bai I. Classify the characters of one line.
#
# C++ traps you with cin >> s, which stops at the first space. Python's equivalent is
# input().split(), same wound.
#
# But Python adds a second trap the statement quietly sets up: the line may be EMPTY, zero
# characters. input() on an empty final line raises EOFError, so the shortest correct
# looking program crashes on exactly the edge case the limits announce.
#
# str.isupper() and friends are not the right tools either: they are Unicode aware, so a
# non-ASCII letter would be counted as a letter. The statement says printable ASCII, and
# the byte ranges say what the statement says.
import sys

def main():
    dong = sys.stdin.buffer.readline().rstrip(b"\r\n")
    hoa = thuong = so = khac = 0
    for c in dong:
        if 65 <= c <= 90:
            hoa += 1
        elif 97 <= c <= 122:
            thuong += 1
        elif 48 <= c <= 57:
            so += 1
        else:
            khac += 1
    print(hoa, thuong, so, khac)

main()
```

Code bài J

```
# Bai J. Simulate a queue.
#
# The C++ version of this problem has no performance trap at all: std::queue is the
# obvious tool and it is O(1). Python has a real one, and it is the single most useful
# thing in this whole session.
#
# A list looks like a queue. hang.append(ten) is O(1), but hang.pop(0) moves every
# remaining element down one slot, so it is O(n). With 2*10^5 OUT operations that is
```

```
# 2*10^10 element moves, and the program that reads correctly is quadratic.
#
# collections.deque pops from the left in O(1). Same five lines, different complexity.
import sys
from collections import deque

def main():
    du_lieu = sys.stdin.buffer.read().split(b"\n")
    n = int(du_lieu[0])
    hang = deque()
    ra = []
    for i in range(1, n + 1):
        d = du_lieu[i].split()
        if not d:
            continue
        if d[0] == b"IN":
            hang.append(d[1])
        elif d[0] == b"OUT":
            ra.append(hang.popleft() if hang else b"EMPTY")
        else:
            ra.append(str(len(hang)).encode())
    sys.stdout.buffer.write(b"\n".join(ra) + b"\n")

main()
```